

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

```
import pandas as pd
import numpy as np
import os, pathlib
from pathlib import Path
```

```
block_df = pd.read_parquet('hbblock_df.parquet')
```

```
exp_block_df = compact_to_expanded(
    block_df, timeseries_col='energy_consumption',
    static_cols=['frequency', 'series_length', 'stdorToU', 'Acorn',
                 'Acorn_grouped'],
    time_varying_cols=['pressure', 'apparentTemperature', 'windSpeed',
                       'precipType', 'icon', 'humidity', 'summary'],
    ts_identifiser='LCLid')
```

```
exp_block_df.head(1)
```

	timestamp	LCLid	energy_consumption	frequency	series_length	stdorToU	Acorn	Acorn_grouped	pressure
0	2012-10-13	MAC000002	0.263	30min	24144	Std	ACORN-A	Affluent	100

Train, Test and Validation Sets

```
test_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==2)
validation_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==1)

train = exp_block_df[~(test_mask | validation_mask)]
validation = exp_block_df[validation_mask]
test = exp_block_df[test_mask]
train.shape, validation.shape, test.shape
```

((11855088, 15), (595200, 15), (518400, 15))

Baseline

```
train_df = train[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
validation_df = validation[
    ['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
test_df = test[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
```

```
train_df.timestamp.min(), train_df.timestamp.max(), \
test_df.timestamp.min(), test_df.timestamp.max(), \
validation_df.timestamp.min(), validation_df.timestamp.max()
```

(Timestamp('2012-01-01 00:00:00'),
Timestamp('2013-12-31 23:30:00'),
Timestamp('2014-02-01 00:00:00'),
Timestamp('2014-02-27 23:30:00'),
Timestamp('2014-01-01 00:00:00'),
Timestamp('2014-01-31 23:30:00'))

```
freq = train_df.iloc[0]['frequency']
timeseries_train = train_df.loc[train_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
timeseries_validation = validation_df.loc[validation_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_
timeseries_test = test_df.loc[test_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
```

```
ts = SeasonalInterpolation(seasonal_period=48*7).fit_transform(ts_df.energy_consumption.values.reshape(-1,1
```

```
ts_df = exp_block_df[exp_block_df['LCLid'] == 'MAC000322'].set_index('timestamp')

ts_df['weekday_name'] = ts_df.index.day_name()
ts_df['weekday'] = ts_df.index.weekday
ts_df['week'] = ts_df.index.isocalendar().week
ts_df['hour'] = ts_df.index.hour
ts_df['month'] = ts_df.index.month
ts_df['day'] = ts_df.index.day
ts_df['month_name'] = ts_df.index.month_name()
ts_df['year'] = ts_df.index.year

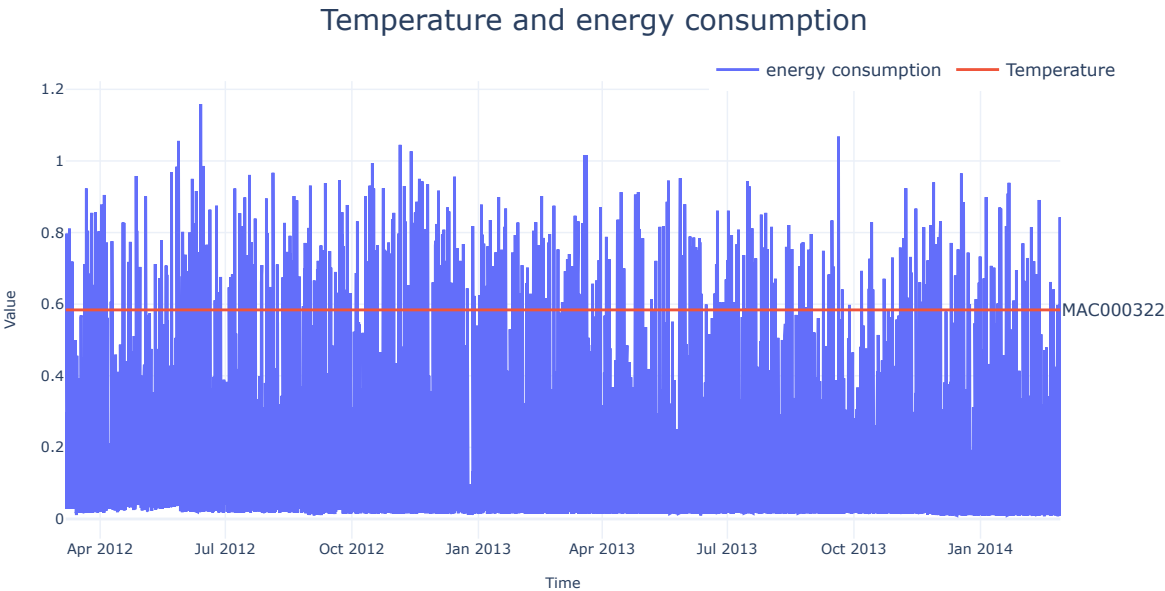
ts_df['month_name'] = pd.Categorical(ts_df['month_name'],
    categories=['January', 'February', 'March', 'April', 'May', 'June',
                'July', 'August', 'September', 'October', 'November', 'December'])
ts_df['weekday_name'] = pd.Categorical(ts_df['weekday_name'],
    categories=['Monday', 'Tuesday', 'Wednesday', 'Thursday',
                'Friday', 'Saturday', 'Sunday'])
```

```
import warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)
warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=UserWarning)
```

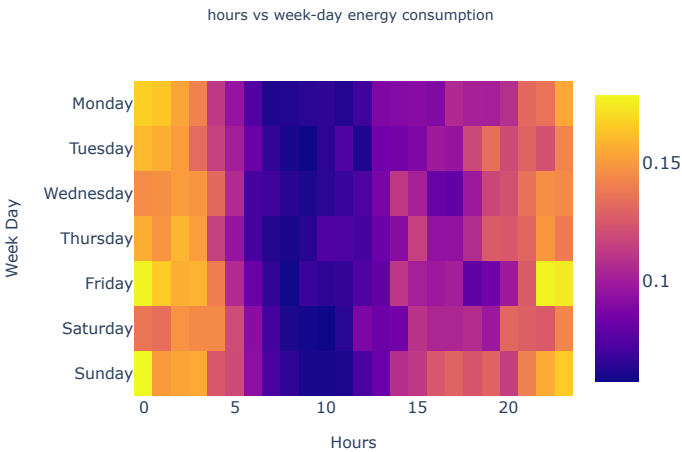
```
ts_df['energy_consumption'] = SeasonalInterpolation(
    seasonal_period=48*7).fit_transform(ts_df['energy_consumption'].values.reshape(-1, 1))

figure = px.line(ts_df, y='energy_consumption', title='Energy consumption for MAC005492')
figure
```

```
''' multi-variate line chart '''
figure__ = two_line_plot_secondary_axis(x=ts_df.index,
    y1 = ts_df.energy_consumption, y2 = ts_df.LCLid,
    y1_name = 'energy consumption', y2_name = 'Temperature',
    title = 'Temperature and energy consumption'
)
figure__.update_layout(
    legend=dict(orientation = 'h', yanchor = 'bottom', y = .98, xanchor = 'right', x = 1,),
    yaxis = dict( #title_text = ylabel,
        titlefont = dict(size = 10), tickfont = dict(size = 10),),
    xaxis = dict( #title_text = xlabel,
        titlefont = dict(size = 10), tickfont = dict(size = 10),)
)
figure__.show()
```

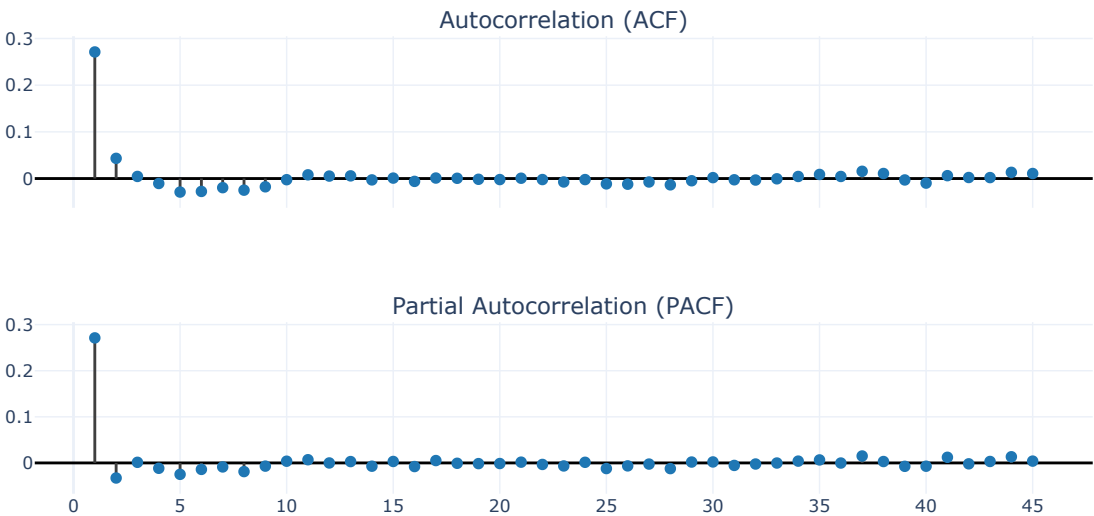


```
''' calendar heatmap '''
plot_df = pd.pivot_table(ts_df, index='weekday_name', values='energy_consumption',
    columns='hour', aggfunc='mean')
figure_ = px.imshow(plot_df, height=600, title='hours vs week-day energy consumption')
figure_ = format_plot(figure_, ylabel='Week Day', xlabel='Hours')
figure_.show()
```



```
''' auto-correlation plots '''
stl = MultiSeasonalDecomposition(seasonal_model='fourier',
    seasonality_periods=['day_of_year', 'day_of_week', 'hour'], model = 'additive',
    n_fourier_terms =10)
res = stl.fit(ts_df['energy_consumption'])

figure_ = plot_autocorrelation(res.resid, vertical=True)
figure_.show()
```



Outlier Detection and Treatment

```
res_df = pd.DataFrame(columns=['# of Outliers', '% of Outliers'])
```

```
def detect_outlier_sd(ts, sd_multiple=2):
    mean = ts.mean()
    std = ts.std()
    higher_nound = mean + sd_multiple*std
    lower_nound = mean - sd_multiple*std
    outlier_mask = (ts > higher_nound) | (ts < lower_nound)
    return outlier_mask

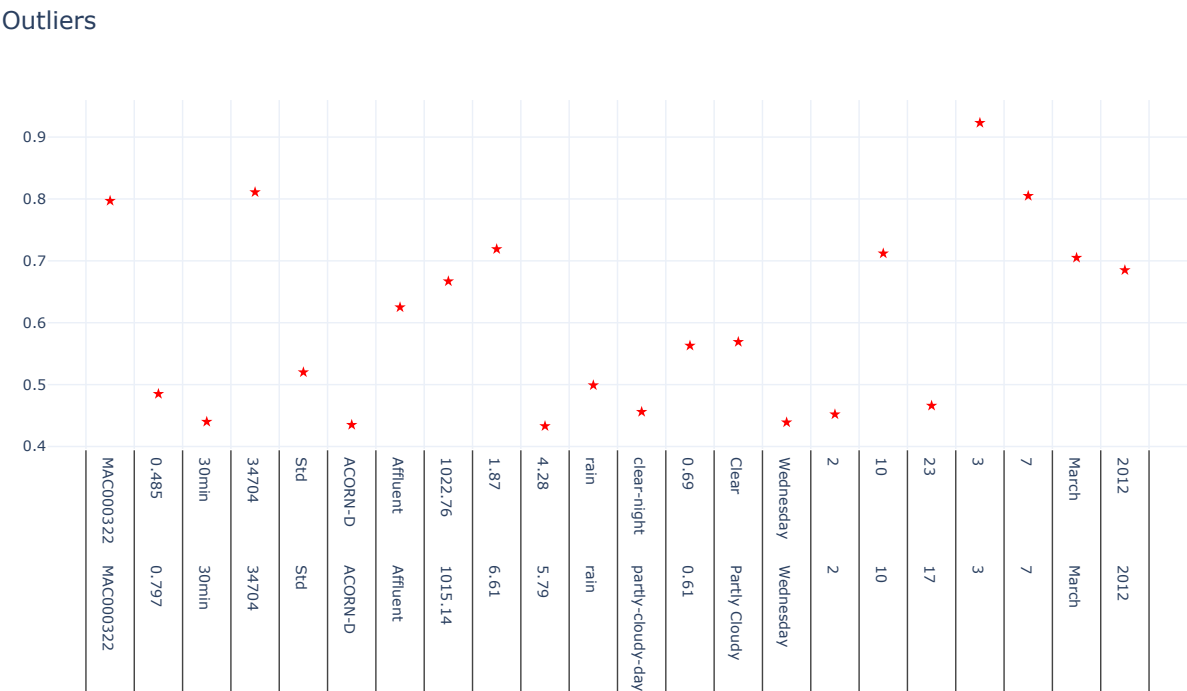
def detect_outlier_iqr(ts, iqr_multiple=2):
    q1, q2, q3 = np.quantile(
        ts, 0.25), np.quantile(ts, 0.5), np.quantile(ts, 0.75)
    iqr = q3 - q1
    higher_nound = q3 + iqr_multiple*iqr
    lower_nound = q1 + iqr_multiple*iqr
    outlier_mask = (ts > higher_nound) | (ts < lower_nound)
    return outlier_mask
```

```
import warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)
warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=UserWarning)
```

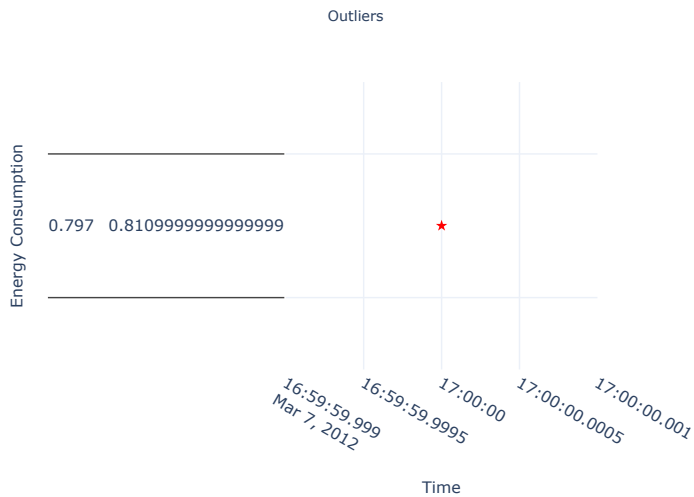
```
''' Fill in missing values '''
ts = SeasonalInterpolation(
    seasonal_period=48*7).fit_transform(ts_df['energy_consumption'].values.reshape(-1, 1))
```

```
outlier_mask = detect_outlier_sd(ts, sd_multiple=3)
res_df.loc['3SD', '# of Outliers'] = outlier_mask.sum()
res_df.loc['3SD', '% of Outliers'] = outlier_mask.sum()/len(ts)*100
res_df
```

```
fig_ = plot_outliers(ts_df, ts, outlier_mask, method='Standard Deviation')
fig_.show()
```



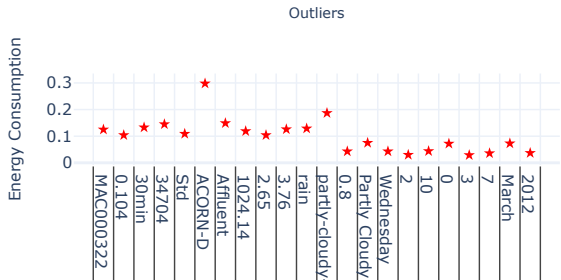
```
outlier_mask = detect_outlier_isolation_forest(ts, outlier_fraction=0.01)
res_df.loc["Isolation Forest", "# of Outliers"] = outlier_mask.sum()
res_df.loc["Isolation Forest", "% of Outliers"] = outlier_mask.sum()/len(ts)
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Isolation Forest")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



```
outlier_mask = detect_outlier_iqr(ts,4)
res_df.loc['4IQR', '# of Outliers'] = outlier_mask.sum()
res_df.loc['4IQR', '% of Outliers'] = outlier_mask.sum()/len(ts)*100
res_df
```

	# of Outliers	% of Outliers
3SD	805	2.319617
Isolation Forest	346	0.00997
4IQR	34333	98.930959

```
fid = plot_outliers(ts_df, ts, outlier_mask, method='IQR')
format_plot(fid, xlabel="Time", ylabel="Energy Consumption")
fid.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.